

Strongly Connected Components & 2-SAT

Lecture 8

CSEN 1038 Advanced Data Structures and Algorithms

Table of contents

1. Motivation: SAT Problems
2. Connected Components Recap
3. Strongly Connected Components
4. Kosaraju's Algorithm
5. 2-SAT

Motivation: SAT Problems

What is SAT?

Boolean Satisfiability Problem (SAT)

Given a Boolean formula in **Conjunctive Normal Form (CNF)**, does there exist an assignment of truth values that makes the entire formula **true**?

$$\underbrace{(x_1 \vee \neg x_2)}_{\text{clause}} \wedge \underbrace{(\neg x_1 \vee x_3)}_{\text{clause}} \wedge \underbrace{(x_2 \vee x_3)}_{\text{clause}}$$

- A **literal**: a variable x_i or its negation $\neg x_i$.
- A **clause**: a disjunction (OR) of literals.
- The formula: a conjunction (AND) of clauses.

- **Combinational Equivalence Checking (CEC):**
Verify two circuit implementations are functionally identical (e.g., pre- vs. post-synthesis in chip design).
- **Formal Verification:** Mathematically prove software/hardware meets its spec — no exhaustive testing needed.
- **Package Managers:** conda and apt use SAT solvers (`libsolv`) to resolve dependency constraints and find a consistent package set.

```
$ conda install numpy  
Solving environment...  
[SAT solver:  libsolv]  
done
```

- k -SAT with $k \geq 3$ is **NP-complete** — no polynomial-time algorithm is known.
- **2-SAT**: every clause has **exactly 2 literals**.
- Solvable in $\mathcal{O}(V + E)$ using **Strongly Connected Components (SCC)**.

Roadmap for This Lecture

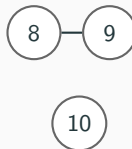
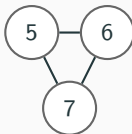
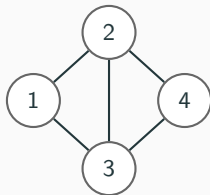
SCC (Kosaraju's Algorithm) $\longrightarrow$ 2-SAT

Connected Components Recap

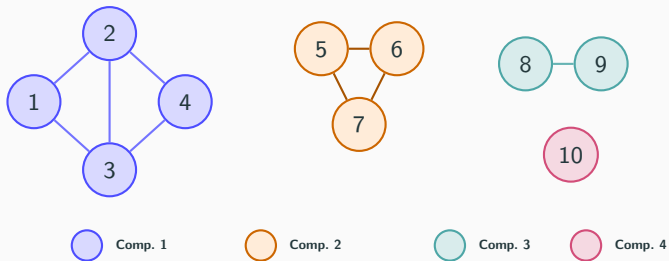
Connected Components in Undirected Graphs

Definition

A **connected component** is a maximal set of vertices where every pair is reachable from one another via undirected paths.



Connected Components — Highlighted



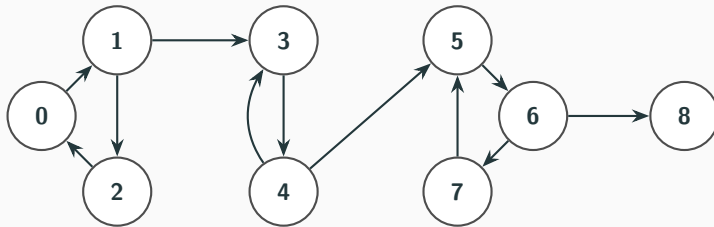
Strongly Connected Components

Strongly Connected Component (SCC)

In a **directed** graph, a maximal set of vertices S such that for every pair $u, v \in S$ there is a directed path from u to v **and** from v to u .

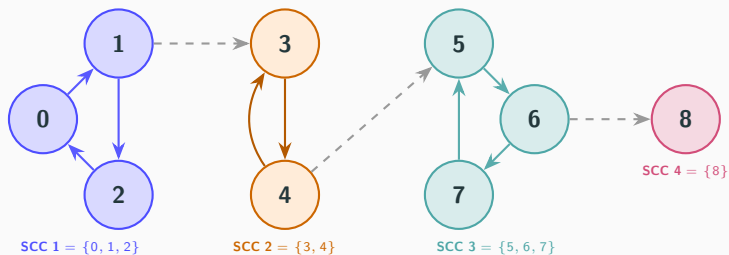
- Every directed graph has a **unique** decomposition into SCCs.
- Collapsing each SCC into a super-node yields the **condensation DAG**.

SCCs — Directed Graph



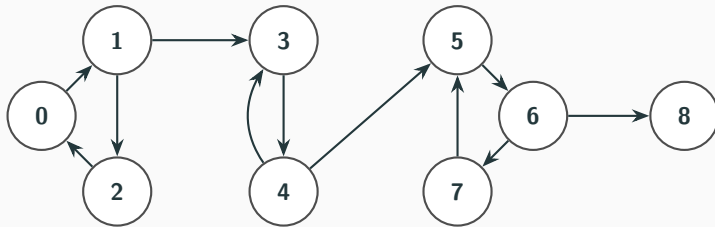
Which groups of nodes can reach each other?

SCCs — Highlighted

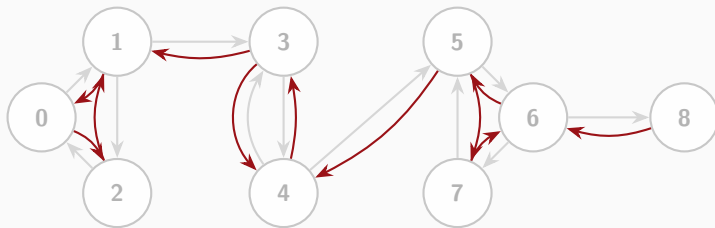


Kosaraju's Algorithm

The Graph



The Reversed Graph G^R



→ Reversed edges (G^R)

→ Original edges (faded, for reference)

Kosaraju's Algorithm

Time: $\mathcal{O}(V + E)$ **Space:** $\mathcal{O}(V + E)$

```
int n;
vector<int> g[MAXN], rg[MAXN]; // original graph and its reverse
vector<int> finish_order;      // DFS finish order (pass 1)
int comp[MAXN];               // SCC id per node
bool visited[MAXN];

// Pass 1: DFS on g -- push each node AFTER all its descendants finish
void dfs_pass1(int v) {
    visited[v] = true;
    for (int u : g[v])
        if (!visited[u]) dfs_pass1(u);
    finish_order.push_back(v);
}

// Pass 2: DFS on rg -- label every reachable node with the same SCC id
void dfs_pass2(int v, int scc_id) {
    visited[v] = true;
    comp[v] = scc_id;
    for (int u : rg[v])
        if (!visited[u]) dfs_pass2(u, scc_id);
}
```

2-SAT

Each clause has **exactly one literal**:

$$x_1 \wedge \neg x_2 \wedge x_3 \wedge \neg x_4 \wedge \dots$$

When is 1-SAT satisfiable?

Each clause has **exactly one literal**:

$$x_1 \wedge \neg x_2 \wedge x_3 \wedge \neg x_4 \wedge \dots$$

When is 1-SAT satisfiable?

Answer

If and only if no variable appears **both** positively (x_i) and negatively ($\neg x_i$).

2-SAT: Definition

Each clause has **exactly two literals**:

$$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee x_3) \wedge \dots$$

Key Insight: Implication Equivalence

$$(a \vee b) \equiv (\neg a \rightarrow b) \equiv (\neg b \rightarrow a)$$

Every clause contributes **exactly two directed implications**.

- Build an **implication graph** with $2n$ nodes.
- Node $i = "x_i \text{ is true}"$ Node $i+n = "x_i \text{ is false}"$
- Run **Kosaraju's SCC** on this graph.

The Satisfiability Condition

After running Kosaraju on the implication graph:

Unsatisfiable

The formula is **UNSAT** iff any variable x_i has its true node and false node in the **same SCC**:

$$\text{comp}[i] = \text{comp}[i + n]$$

An implication loop $x_i \rightarrow \neg x_i \rightarrow x_i$ is a contradiction.

Constructing the Assignment

If SAT, set $x_i = \text{true}$ when its true literal's SCC appears **later** in topological order (higher SCC id from Kosaraju):

$$\text{answer}[i] = (\text{comp}[i] > \text{comp}[i + n])$$

2-SAT — Struct Setup & add_clause

```
struct TwoSAT {
    int n;
    vector<vector<int>> g, rg;    // implication graph + reverse (2n nodes each)
    vector<int> finish_order;
    vector<int> comp;           // comp[v] = SCC id of node v
    vector<bool> visited;

    TwoSAT(int n)
        : n(n), g(2*n), rg(2*n), comp(2*n), visited(2*n) {}

    // Add directed edge u->v and maintain its reverse for Kosaraju pass 2
    void add_edge(int u, int v) {
        g[u].push_back(v);
        rg[v].push_back(u);
    }

    // Clause: (x_i = f) OR (x_j = h)
    //   node i   = "x_i is true"   |   node i+n = "x_i is false"
    //   f=true   -> positive literal (x_i)
    //   f=false  -> negative literal (not x_i)
    void add_clause(int i, bool f, int j, bool h) {
        add_edge(i + (f ? n : 0), j + (h ? 0 : n)); // neg(xi=f) -> (xj=h)
        add_edge(j + (h ? n : 0), i + (f ? 0 : n)); // neg(xj=h) -> (xi=f)
    }
}
```

2-SAT — Kosaraju + solve

Time: $\mathcal{O}(V + E)$ ($V=2n, E=2m$) **Space:** $\mathcal{O}(V + E)$

```
void dfs1(int v) {           // identical to Kosaraju pass 1
    visited[v] = true;
    for (int u : g[v])
        if (!visited[u]) dfs1(u);
    finish_order.push_back(v);
}

void dfs2(int v, int id) { // identical to Kosaraju pass 2
    visited[v] = true;  comp[v] = id;
    for (int u : rg[v])
        if (!visited[u]) dfs2(u, id);
}

// Returns true iff satisfiable; answer[i] holds the assigned value of x_i
bool solve(vector<int>& answer) {
    fill(visited.begin(), visited.end(), false);
    for (int i = 0; i < 2*n; i++)
        if (!visited[i]) dfs1(i);           // Kosaraju pass 1

    fill(visited.begin(), visited.end(), false);
    reverse(finish_order.begin(), finish_order.end());
    int id = 0;
    for (int v : finish_order)
        if (!visited[v])
            dfs2(v, id++);                 // Kosaraju pass 2

    answer.resize(n);
    for (int i = 0; i < n; i++) {
        if (comp[i] == comp[i + n]) return false; // contradiction
        answer[i] = (comp[i] > comp[i + n]);      // later SCC = chosen value
    }
    return true;
}

};
```


Example: Holiday Party Problem

Setup: n couples; invite exactly one person per couple (A_i or B_i). Some guests conflict.

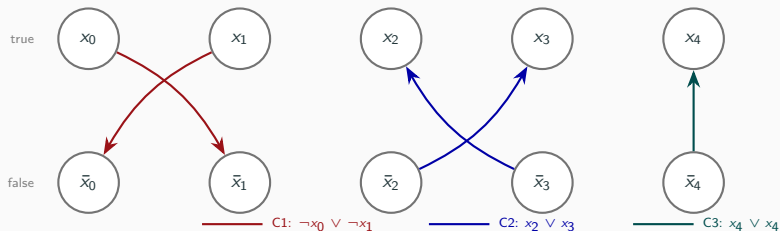
Let $x_i = \text{true}$ mean “invite A_i ”, $x_i = \text{false}$ mean “invite B_i ”.

- A_0 and A_1 cannot both attend: $\neg x_0 \vee \neg x_1$
- A_2 or A_3 must attend: $x_2 \vee x_3$
- Force A_4 to attend: $x_4 \vee x_4$ (self-clause)

```
TwoSAT sat(n);
sat.add_clause(0, false, 1, false); // neg(x0) OR neg(x1)
sat.add_clause(2, true, 3, true);   // x2 OR x3
sat.add_clause(4, true, 4, true);   // x4 (forced via self-clause)
```

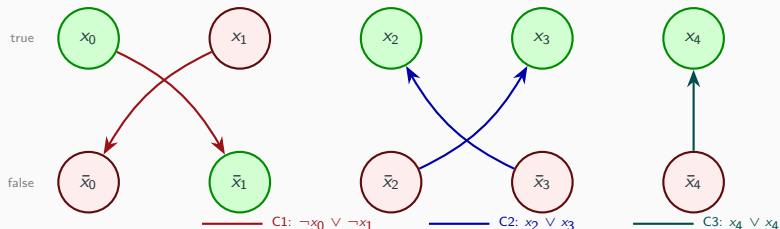
Holiday Party — Implication Graph

Edges follow from the two implications of each clause: $(a \vee b) \equiv (\neg a \rightarrow b) \equiv (\neg b \rightarrow a)$



Holiday Party — Implication Graph

Edges follow from the two implications of each clause: $(a \vee b) \equiv (\neg a \rightarrow b) \equiv (\neg b \rightarrow a)$



Chosen (higher SCC id): $x_0=T, x_1=F, x_2=T, x_3=T, x_4=T \Rightarrow$ Invite A_0, B_1, A_2, A_3, A_4

Questions?

- <https://youtu.be/jDi0X7KvZnE?t=3843>
- <https://codeforces.com/blog/entry/92977>
- <https://codeforces.com/blog/entry/16205>
- <https://cp-algorithms.com/graph/strongly-connected-components.html>